

SI 100B Programming Assignment 3

SI 100B TA Team

October 31, 2025

Notice

- This homework is due **21:00 November 6, Thursday**, please start early.
- The problems should be solved **individually**. You should submit solutions for all problems to the **online judge** (OJ) system, which will give a score for each submission automatically.
- You may discuss course material, programming languages, general techniques, and share test cases with others.
- You're encouraged to ask any question about problem description, environment setup, error information, grammar, functions, methods etc., or post general ideas at Piazza. It can help others too!
- However, you may **NOT** read, possess, copy, or submit any part of other's solution. You may not ask others to directly debug your program or provide specific lines of solution. You should not access another person's account. Also, do not offer such help, and protect your account and code. Giving or receiving inappropriate help is **plagiarism**, an infringement of academic integrity.
- You are not allowed to write your solution directly by or based on code generated by **AI tools**.
- **Good luck and have fun!**

1 Nested List Flattener (20 points)

1.1 Background

You are developing a tool to process complex nested data structures. The tool needs to handle nested lists of arbitrary depth and flatten them into one-dimensional lists. This assignment focuses on implementing recursive algorithms to process such nested structures.

1.2 Description

Implement a function that can flatten nested lists of arbitrary depth into a one-dimensional list.

```
def flatten_nested_list(nested):  
    """  
    Flatten a nested list of arbitrary depth into a one-dimensional list.  
  
    Args:  
    nested_list: A nested list structure that can contain integers, strings,  
    floats, booleans, or other nested lists  
  
    Returns:  
    A flattened one-dimensional list whose elements are in the same order  
    as they appear from left to right in the printed original list.  
    """  
    pass
```

1.2.1 Data Structure

- Handle nested lists of arbitrary depth.
- List elements can be: integers, strings, floats, booleans, or other nested lists.

1.2.2 Processing Rules

- The flattened list should maintain the element order following the printing order of the elements in the original list.
- Empty lists should flatten to empty lists.
- All non-list elements (integers, strings, floats, booleans) should be directly included in the result.
- Relative order of elements should be preserved.

1.3 Submission

Please submit **ONLY your** `flatten_nested_list` **functions**. Do not submit any other code used for local testing.

In example, input is a string representing a nested list, and output is the flattened list.

1.4 Examples

1.4.1 Example 1

- **Input:**

```
[1, [2, [3, 4], 5], 6, [7, 8]]
```

- **Output:**

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

1.4.2 Example 2

- **Input:**

```
["hello", [1, "world", [2.5, True, [3]]], 4]
```

- **Output:**

```
["hello", 1, "world", 2.5, True, 3, 4]
```

1.4.3 Example 3

- **Input:**

```
[]
```

- **Output:**

```
[]
```

1.4.4 Example 4

- **Input:**

```
[1, 2, 3]
```

- **Output:**

```
[1, 2, 3]
```

1.4.5 Example 5**• Input:**

```
[[[[["deep"]]]], "shallow"]
```

• Output:

```
["deep", "shallow"]
```

2 Sublist Operation (40 points)

2.1 Background

To test your list-handling skills, the cute maiden Yakumo Yukari decides to ask you to implement three operations related to sublists.

2.2 Description

You are asked to implement three functions. Each function should support two semantics: in-place modification or returning a copy (See Processing Rules for details).

1. Implement a function `reverse_sublist` that reverses the elements in a sublist defined by the half-open interval $[l, r)$ of a list `lst`.

Reverse: Reorder the elements of a list so that their sequence is inverted — the first element becomes the last, the second becomes the second-to-last, and so on.

Example: For a list `[1, 2, 3, 4, 5, 6]`, reversing it yields `[6, 5, 4, 3, 2, 1]`.

```
def reverse_sublist(lst, l, r, inplace):
    """
    Reverse the elements in the sublist [l:r] of a list.

    Args:
        lst (list): The input list to be processed.
        l (int): The starting index of the sublist (inclusive).
        r (int): The ending index of the sublist (exclusive).
        inplace (bool):
            If True, modifies the list in place and returns the same list.
            If False, returns a new list with the specified sublist reversed.

    Returns:
        list: The list with the specified sublist reversed.

    Raises:
        AssertionError: If l or r is out of range, or if l >= r.
    """
    pass
```

2. Implement a function `rotate_sublist` that rotates a specified sublist within the half-open interval $[l, r)$ of a list `lst` to the left by k positions.

Rotate left: Move each element of a list of length n to the left by k positions, wrapping the first k elements around to the end. So in fact, k can be any integer by performing modulo operation.

Example: For $n = 6$ and $k = 2$, rotating $[1, 2, 3, 4, 5, 6]$ left yields $[3, 4, 5, 6, 1, 2]$.

```
def rotate_sublist(lst, l, r, k, inplace):
    """
    Rotate the elements in the sublist [l:r] of a list to the left by k positions.

    Args:
        lst (list): The input list to be processed.
        l (int): The starting index of the sublist (inclusive).
        r (int): The ending index of the sublist (exclusive).
        k (int): The number of positions to rotate left. Can be any integer.
        inplace (bool):
            If True, modifies the list in place and returns the same list.
            If False, returns a new list with the specified sublist rotated.

    Returns:
        list: The list with the specified sublist rotated left by k positions.

    Raises:
        AssertionError: If l or r is out of range, or if l >= r.
    """
    pass
```

3. Implement a function `swap_sublist` that swaps two sublists of a list `lst`, defined by intervals $[l_1, r_1)$ and $[l_2, r_2)$.

```
def swap_sublist(lst, l1, r1, l2, r2, inplace):
    """
    Swap two sublists [l1:r1] and [l2:r2] of a list.

    Args:
        lst (list): The input list to be processed.
        l1 (int): The starting index of the first sublist (inclusive).
        r1 (int): The ending index of the first sublist (exclusive).
        l2 (int): The starting index of the second sublist (inclusive).
        r2 (int): The ending index of the second sublist (exclusive).
        inplace (bool):
            If True, modifies the list in place and returns the same list.
            If False, returns a new list with the specified sublists swapped.
    """
    pass
```

```

Returns:
    list: The list with the specified sublists swapped.

Raises:
    AssertionError:
        If the sublist indices are out of range,
        if l1 >= r1 or l2 >= r2,
        if the two sublists are not of the same length
        or the two sublists overlap.
"""
pass

```

2.2.1 Processing Rules

- Note that the interval is half-open $[l, r)$: it includes the element at index l but excludes the element at index r , which is the same behavior as Python slicing. When $r = \text{len}(\text{lst})$, it indicates up to the end of the list.
- For inplace:
 - When `inplace=True`, the function should modify the passed list in-place and **return the same list object** (the caller's list will be modified).
 - **IMPORTANT:** When `inplace=True`, if you don't return the same list object, you will get RE (Runtime Error).
 - When `inplace=False`, the function should not change the original list and return a new modified list (**shallow copy** of the outer list; For the testcases of this problem, **all elements in the list are immutable**)
- For assertion:
 - For the three functions, you need to assert l and r in the range of `lst` and $l < r$, with message `Invalid sublist`.
 - For `swap_sublist`, you need to also assert the two sublists have the same length and no overlap, with message `Invalid sublist`.
 - **IMPORTANT:** If you throw wrong exception or with wrong message, you will get RE (Runtime Error).

2.2.2 Hint

- You only need to make the most of list slicing and concatenation operations, while also paying attention to the copying behavior associated with list slicing.
- (EXTRA: If you have the time and are willing to think about it, you can consider and try to implement `rotate_sublist` and `swap_sublist` by using `reverse_sublist`. This will NOT affect the final grading.)

2.3 Submission

Please submit **ONLY your** `reverse_sublist`, `rotate_sublist` and `swap_sublist` **functions**. Do not submit any other code used for local testing.

In example, input is a series of codes, and output is the output of the codes.

2.4 Examples

2.4.1 Example 1

• **Input:**

```
lst = [1, 1, 4, 5, 1, 4]
print(reverse_sublist(lst, 0, 3, False))
print(lst)
print(reverse_sublist(lst, 4, 6, True))
print(lst)
print(rotate_sublist(lst, 0, 3, 1, False))
print(lst)
print(swap_sublist(lst, 1, 2, 2, 3, True))
print(lst)
print(swap_sublist(lst, 0, 2, 4, 6, False))
print(lst)
```

• **Output:**

```
[4, 1, 1, 5, 1, 4]
[1, 1, 4, 5, 1, 4]
[1, 1, 4, 5, 4, 1]
[1, 1, 4, 5, 4, 1]
[1, 4, 1, 5, 4, 1]
[1, 1, 4, 5, 4, 1]
[1, 4, 1, 5, 4, 1]
[1, 4, 1, 5, 4, 1]
[4, 1, 1, 5, 1, 4]
[1, 4, 1, 5, 4, 1]
```

2.4.2 Example 2

- **Input:**

```
lst = [1, 1, 4, 5, 1, 4]
print(rotate_sublist(lst, 1, 5, 2, False))
print(rotate_sublist(lst, 1, 5, 3, False))
print(rotate_sublist(lst, 1, 5, 1919810, False))
print(rotate_sublist(lst, 2, 6, 1, True))
print(rotate_sublist(lst, 2, 6, 1, True))
print(rotate_sublist(lst, 2, 6, 1, True))
print(rotate_sublist(lst, 2, 6, 1, True))
print(rotate_sublist(lst, 1, 5, -1919810, True))
```

- **Output:**

```
[1, 5, 1, 1, 4, 4]
[1, 1, 1, 4, 5, 4]
[1, 5, 1, 1, 4, 4]
[1, 1, 5, 1, 4, 4]
[1, 1, 1, 4, 4, 5]
[1, 1, 4, 4, 5, 1]
[1, 1, 4, 5, 1, 4]
[1, 5, 1, 1, 4, 4]
```

2.4.3 Example 3

- **Input:**

```
lst = [1, 1, 4, 5, 1, 4]
# These codes will fail the asserts
reverse_sublist(lst, -1, 5, False)
reverse_sublist(lst, 1, 8, True)
reverse_sublist(lst, 6, 2, False)
rotate_sublist(lst, 1, 9, 2, True)
swap_sublist(lst, 2, 1, 5, 4, False)
swap_sublist(lst, 0, 2, 4, 5, True)
swap_sublist(lst, 0, 3, 2, 5, True)
```

- **Output:**

```
AssertionError: Invalid sublist
```

2.5 Data Constraints

In this problem, all elements in the list are immutable.

In this problem, the test data includes a total of 11 subtasks. Each subtask contains several test cases, and you will receive the full score for a subtask if and only if you pass **all the test cases within that subtask**. For easier debugging, here are the data constraints for each subtask:

- Subtask 1 (5% pts): Only `reverse_sublist`, $0 \leq l < r \leq \text{len}(\text{lst})$ and `inplace=False`.
- Subtask 2 (5% pts): Only `reverse_sublist`, and $0 \leq l < r \leq \text{len}(\text{lst})$.
- Subtask 3 (10% pts): Only `reverse_sublist`.
- Subtask 4 (5% pts): Only `rotate_sublist`, $0 \leq l < r \leq \text{len}(\text{lst})$, $0 < k < r - l + 1$ and `inplace=False`.
- Subtask 5 (5% pts): Only `rotate_sublist`, $0 \leq l < r \leq \text{len}(\text{lst})$ and `inplace=False`.
- Subtask 6 (5% pts): Only `rotate_sublist`, and $0 \leq l < r \leq \text{len}(\text{lst})$.
- Subtask 7 (10% pts): Only `rotate_sublist`.
- Subtask 8 (5% pts): Only `swap_sublist`, and $0 \leq l_{1,2} < r_{1,2} \leq \text{len}(\text{lst})$, the two sublists have the same length and no overlap, and `inplace=False`.
- Subtask 9 (5% pts): Only `swap_sublist`, and $0 \leq l_{1,2} < r_{1,2} \leq \text{len}(\text{lst})$, the two sublists have the same length and no overlap.
- Subtask 10 (10% pts): Only `swap_sublist`.
- Subtask 11 (35% pts): No specific constraints.

3 Document Keyword (40 points)

3.1 Background

In many modern information retrieval systems — from search engines to news recommendation systems — understanding the *main topic* of a document is crucial. One classic way in traditional Natural Language Processing to measure the importance of a word in a document is through the **TF-IDF** (Term Frequency – Inverse Document Frequency) score.

TF-IDF assigns a high weight to words that appear frequently in a document but rarely across other documents. This helps highlight unique or topic-specific terms that represent the document's main content. In this problem, you will practice with dictionaries to analyze multiple texts and find their most representative keywords using TF-IDF.

3.2 Description

You will be given n text documents. Your goal is to compute the TF-IDF score for each word in each document and then identify the **word with the highest TF-IDF score** in that document. This word will serve as the document's *keyword* or *topic word*. You will then output the keyword and its TF-IDF score for each document.

To compute the TF-IDF, you need to follow these steps:

1. **Compute TF (Term Frequency)** for each word in a document:

$$\text{TF}_{t,d} = \log_{10} (\text{count}(t, d) + 1)$$

where $\text{count}(t, d)$ is the number of times word t occurs in document d .

2. **Compute IDF (Inverse Document Frequency)** across all documents:

$$\text{IDF}_t = \log_{10} \left(\frac{N}{\text{DF}_t} \right)$$

where DF_t is the number of documents that word t occurs in. N is the total number of documents in the collection.

3. **Combine TF and IDF** to obtain the **TF-IDF** score for each word in each document:

$$\text{TF-IDF}_{t,d} = \text{TF}_{t,d} \times \text{IDF}_t$$

After computing the TF-IDF for each word in each document, you need to find the **word with the highest**

TF-IDF score in each document and output both the word and its TF-IDF score (rounded to four decimal places). If two words have the same TF-IDF score, choose the one that appears earlier in the document.

Note: You may either **implement all functions on your own** or **use the provided** `template.py`, which contains function stubs and guiding comments. You are free to follow its structure or write your own implementation as long as your I/O behavior meets the specification.

3.3 Format

3.3.1 Input

The first line of input contains a single integer n — the number of test cases. The next n lines, each line is a document, which contains an indefinite number of words, separated by spaces, all **ONLY** consisting of lowercase letters.

3.3.2 Output

The output contains n lines. The i -th line contains the keyword of the i -th document and the TF-IDF score of the keyword. All floats should be rounded to four decimal places.

3.4 Examples

3.4.1 Example 1

- **Input:**

```
7
miku miku you can call me miku
blue hair blue tie hiding in your wifi
open secrets anyone can find me
hear your music running through my mind
i am on top of the world because of you
all i wanted to do is follow you
i will keep singing along to all of you
```

- **Output:**

```
miku 0.5088
blue 0.4032
open 0.2544
hear 0.2544
of 0.2596
wanted 0.2544
will 0.2544
```

3.4.2 Example 2**• Input:**

```
10
ee oo ee ee ee oo
oo oo ee oo oo oo oo
ee oo oo oo ee oo oo
oo ee
ee ee oo ee oo ee ee
ee oo ee oo oo
ee ee ee
oo oo
oo oo ee ee oo ee
ee oo ee ee ee
```

• Output:

```
ee 0.0320
oo 0.0387
oo 0.0356
oo 0.0138
ee 0.0356
oo 0.0275
ee 0.0275
oo 0.0218
oo 0.0275
ee 0.0320
```

3.4.3 Example 3

- **Input:**

```
19
and the universe said i love you
and the universe said you have played the game well
and the universe said everything you need is within you
and the universe said you are stronger than you know
and the universe said you are the daylight
and the universe said you are the night
and the universe said the darkness you fight is within you
and the universe said the light you seek is within you
and the universe said you are not alone
and the universe said you are not separate from every other thing
and the universe said you are the universe tasting itself talking to itself reading its
    own code
and the universe said i love you because you are love
and the game was over and the player woke up from the dream
and the player began a new dream
and the player dreamed again dreamed better
and the player was the universe
and the player was love
you are the player
wake up
```

- **Output:**

```
i 0.2943
have 0.3849
everything 0.3849
stronger 0.3849
daylight 0.3849
night 0.3849
darkness 0.3849
light 0.3849
alone 0.3849
separate 0.3849
itself 0.6101
because 0.3849
over 0.3849
began 0.3849
dreamed 0.6101
was 0.2413
was 0.2413
player 0.1507
wake 0.3849
```